Manil Bastola
CSE566S
Final Project Report

A critique on "High-Performance Computing with Quantum Processing Units**"**

1. Introduction

The clash of Moore's Law with the laws of Physics has brought challenges and opportunities in the field of computing. The biggest challenge has been to reduce the size of transistor further than 1nm. This is since a tenth of a nm pertains to the atomic scale whereby reliably controlling the electron flow becomes impossible due to quantum effects, such as electrons jumping outside of the transistor, etc. So far, in High Performance Computing (HPC), Moore's law has been maintained using combination of strategies - superscalar instructions, multi-processor/threads and GPU technologies, etc. However, Lucas, et. al notes the difficulties with conventional methods in reaching the exascale target in computing [2]. They argue the need for revolutionary approaches but highlight that revolutionary approaches might face challenges in practical integration.

The paper "High-Performance Computing with Quantum Processing Units" by Britt et. al.[1] addresses challenges of integrating Quantum Computing (QC) in a HPC platform. The paper begins with introducing Quantum Computing as one such revolutionary approach in realizing HPC beyond current exascale targets. The paper then outlines physical and functional design requirements of an integrated QC system using Quantum Processing Units (QPUs) and identify two integration approaches. The paper suggests the usage of a "quantum interconnect" that would be significant in both the integration approaches for full realization of the potential a QPU integrated HPC system. Finally, the paper outlines the shortcomings of current metrics such as FLOPS for benchmarking the QPU systems and provide alternative metric for consideration. Henceforth in this paper, the term "the paper" will correspond to Britt et. al.'s paper.

2. Core Component Analysis

In this section, we outline the core components of the paper. Such core components reflect the key ideas put forward by the paper in HPC and QC integration. In doing so, we analyze the paper's effectiveness in conveying the material presented in an Analysis subsection.

2.1 The QPU

A simplest CPU architecture comprises of a processor and memory with data-buses and control units between the two. A processor consists of registers and ALU. The paper outlines an architecture for a quantum processing unit (QPU). Here, a simplest quantum processor consists of a single quantum register. A quantum register is defined as an addressable array of a two-level (|0> and |1>[2]) quantum physical system. The size of register depends on *n*-qubits that it can store. A qubit in a two-level system consists of values 0, 1 or a combination of the two. For example, a 2-qubit register can represent states

[1] Keith A. Britt, PHD candidate in the field of Quantum Computing at University of Tennessee.
Travis S. Humble, Senior Scientist at Oak Ridge National Laboratory and Director of the Quantum Computing Institute.

[2] |*x*> represents Dirac notation for a quantum state of its classical analog *x*.

|00>, |01>, |10> and |11> simultaneously with equal probabilities. Hence, although the *n*-qubit system can represent 2^n values like any classical system, it can realize those values all at once, which a classical system cannot. This is termed as the "inherent parallelism", which is the basis of the excitement behind quantum computing.

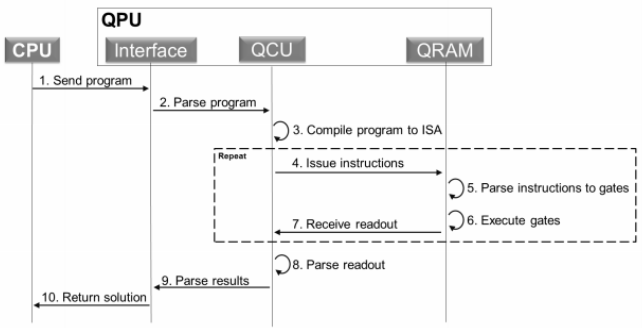


Fig 1: A sequence diagram modeling QPU-CPU interactions utilizing QRAM model proposed by Britt et. al's paper.

The paper then outlines QRAM model. In the QRAM model, a QPU operation begins with preparing the quantum register in a well-defined initial state and then acting sequence of gates that may act on individual or multiple register elements summarized in the Fig 1. Additionally, the paper notes a need for an interface which may be implemented in hardware or software for control unit. The paper seems to be arguing for a HDL-like interface when they argue that the interface would provide exposure to the gates and registers needed for programming low-level quantum algorithms. Additionally, the paper mentions that masking quantum hardware details for a software developer being important. Finally, the paper argues for a compiler that would avoid the need to for APIs that integrate quantum processing instructions directly into the codebase.

Analysis:

This section is highly simplistic in its QPU integration scheme. For example, the paper takes QRAM and quantum registers, important components of the QPU, as black-boxes for the overall Quantum Instruction Set Architecture (ISA). This makes the paper easy to comprehend. Additionally, we understand that the focus of the paper is HPC integration and not QPU design. However, the paper could have provided good references on the details or the current works being done on such quantum architectures. Additionally, there is no mention of ALU and memory, integral aspects of a processor, in this section. The paper hints using series of quantum gates for accomplishing computation but never explicitly mentions them as classical ALU analogs. Upon investigating on the QRAM model in El houshi et. al.'s paper [3] we understood the QRAM model as a combination of such quantum gates (qALU) along with a memory system both controlled by classical control units. Although Fig 1 in the paper demonstrates such a system, explicitly displaying memory and ALU would have made the diagram computer engineering specific rather than algorithmic. In contrast, [3] provides detailed description of the qALU+memory subsystem.

2.2 Quantum Interconnect

Though the "quantum interconnect" is discussed in the QPU section, it is one of the key ideas put forward by the paper and hence, requires separate analysis. The interconnect plays important role in

HPC-QPU integration. This is where the paper deviates from the QRAM model incorporating multiple QPU's that interact with each other in a quantum network. The quantum interconnect is similar to classical interconnect except that the registers in two nodes can be in an entangled state. Because of entanglement between the nodes, for a quantum interconnect featuring q nodes and an n-qubit register in each node can operate in the dimension of 2^{nq} instead of $q2^{n}$, which would be the dimension of a classical interconnect. The only limitation is the "no-cloning" requirement that states that entangled states (intermediate computations) may not be copied during the communication.

The paper then outlines two method for communication: direct transmission and quantum teleportation. The direct transmission is analogous to classical transmission whereby the value of the source register is transferred over the interconnect to the destination QPU, which on successful transmission swaps its register values with the information. Meanwhile, in quantum teleportation, the values of registers can be transferred to a second register without utilizing a physical quantum interconnect. The information transfer is accomplished using pre-existing entanglement between the QPUs. The only physical medium necessary is a classical interconnect to transfer metadata, which is much less in size than the actual information, from the source QPU to the destination. Although teleportation sounds impossible, the reader should note that quantum teleportation has been accomplished for over 100km distances [6].

Analysis

The paper fails to provide an argument for the need of an interconnect. In classical systems, interconnect help communication between networked architectures. The networked computers such as in distributed memory systems are advantageous when data needs distribution due to memory limitations on individual machines, parallelism is bottlenecked by race conditions, algorithms require high scalability, etc. The paper does not address why a single QPU with multiple large qubit sized registers based on the QRAM model would be insufficient for HPC. It seems that the race conditions due to shared memory should not be an issue for quantum computers since the computations are inherently parallel. Additionally, the paper fails to provide appropriate architecture, topology for such interconnected quantum nodes that enhance communication.

The need for quantum interconnect is explained by Spiller et. al. [5]. The need for communication between nodes arises for scalability. This is different from its classical counterpart where more nodes mean more distributed memory such that problems with large inputs can be handled. In quantum computation, the biggest challenge has been to not let the entangled photon states decohere (a phenomenon that destroys entanglement). More photons interacting in a single node increase chances for decoherence and hence, limiting photons on single nodes and communicating intermediate results over network is proposed [5]. Though communicating photon states over interconnect has its limitations, it makes the quantum system viable for large qubit states. As summarized by Kimble: "At any stage in the development of quantum technologies, there will be a largest size attainable for individual quantum processing units, which can be superseded by linking such units together into a fully quantum network" [4]. The lattice topology of such an interconnect is also demonstrated by Kimble [4].

2.3 Integration pathways

The paper proposes two HPC-QC integration pathways – the loose and the tight integration. The loose integration path is a client-server model whereby QPUs remain isolated from the host HPC system and is

connected via a network interface. Since the current QPU devices require special environmental conditions such as ultralow temperatures, ultrahigh vaccum etc., the loose integration approach is practical. However, as the paper points out, the communication between the HPC client and the QC server becomes the primary bottleneck. Hence, there is a trade-off between communication latency and the computational speedup gained from using QPUs instead of CPUs. This implies that loose integration approaches may only be viable in a HPC setting if the computational gains are significant over conventional approaches. The loose integration is depicted in Figure 2 below.

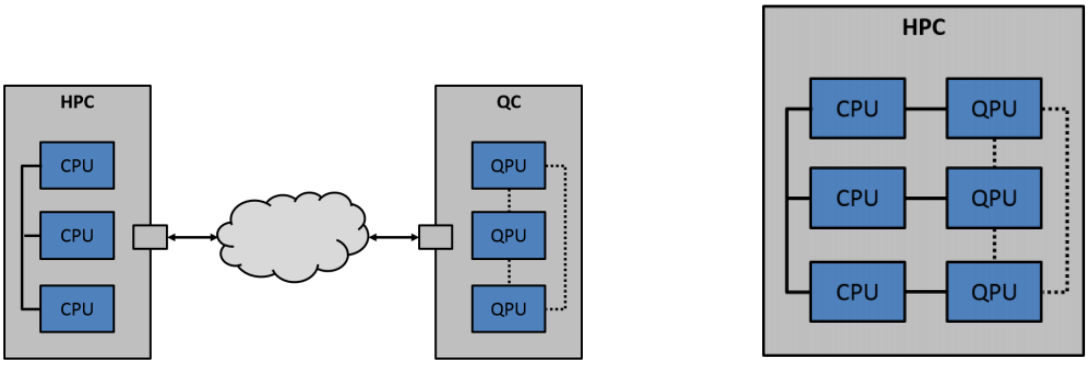


Fig 2: Loose integration pathway (left) and a tight integration model (right) proposed in the Britt et. al's paper.

The tight integration path assumes that physical integration of QCs with HPC is viable. Here, the CPU's are connected to the QPUs using classical channels and are physically close to each other, same motherboard if possible. The paper argues that this approach takes care of latencies related bottlenecks but would require significant leaps in technology. The tight integration can be further abstracted into three pathways as shown in Figure 4 and 5 of the paper. The first case is a shared resource model whereby multiple CPUs are connected to a single QPU. Since the single QPU manages requests from multiple CPUs some sort of scheduling becomes necessary for which the paper argues for a classical controller interface. The second case is multiple-CPUs multiple QPU model where each CPU is connected to a QPU node. The paper argues that such a system if natural since the top-level memory management is driven by domain decomposition for low-level data for single nodes. For example, parallel-computing typically makes use of domain decomposition. However, quantum algorithms avoid such kind of problem partitioning. Hence, the paper acknowledges that this approach restricts the amount of parallelism possible with QCs. Hence, the paper argues for a third approach, which is a multiple-CPUs multiple QPU model but with quantum interconnect connecting the QPUs. The third approach establishes the "best-of-both-worlds" concept and realizes the full quantum potential. The third approach is shown in Fig 2 above.

Analysis:

The QPU-HPC integration methodology proposed are the core strengths of the paper. We appreciate the practicability of the loose integration approach put forward by the paper. This is a MPI-like approach and makes the current QC technology viable for common usage. For example, IBMQ, a cloud based QC server for research and business purposes, not only allows QC job submissions, but also creates VHDL-like codes for learning implementations. Additionally, the paper is also correct in pointing out that the loose integration approach can only be viable if the computational gains are significant over

conventional approaches. For example, the loose integration model can be compared to a cloud based Hadoop Mapreduce framework. Although such a system is highly scalable, it is far from high performing due to latencies involved in communication.

Despite significant latencies involved, we believe that loose integration is still viable for HPC. This is because some of the classes of problems that are considered intractable by conventional approaches can be performed by quantum computers in reasonable amount of time. For example, the traveling salesman problem (TSP) is a NP-complete problem for which if the input size N>300, it would take any conventional computer billions of years to come up with a solution. This is true for any conventional HPC system since we can at best scale performance polynomial order for a conventional HPC system while the problem complexity scales in exponential order. Meanwhile, QCs can theoretically compute TSP solution for N>300 in seconds [7]. For example, in 2012, the D-wave V5 chip with 439 operational qubits found TSP optimal solutions for some large input size in less than half a second. The best classical software solver required 30 minutes to find those same solutions, which put D-wave in at roughly the same rank as the tenth ranked supercomputer in the world as per the November 2012 Top 50 with respect to the TSP [8]. Hence, with more qubits, communication latencies may start becoming insignificant for non-polynomial class problems.

The tight integration approach is similar to CPU-GPU integration. As with GPU, the early integration requires overcoming both hardware and software challenges. We agree with the need of high technological advances in terms of hardware, such as GPU itself and the software, such as CUDA libraries, before tight integration could be realized. However, we find that the paper is making some assumptions regarding the nature of tight integrated HPC. For example, the need for a conventional HPC is not established. An HPC emerging from a cluster of QPUs as opposed to the one integrated with an already established bulky HPC might be more practical. For example, a single CPU/microcontroller for resource allocation and job control may be easier to realize than a multiple CPUs. This may not be possible in room-temperature settings, but virtually all HPC systems are located in a controlled environment of data-centers. However, this comes with challenges of making QPU perform job scheduling and resource allocation. We can find a way around this problem by casting the task at hand in a different form such as a TSP into a simulated annealing problem, which is an embarrassingly parallel problem, which would then utilize inherent parallelism in the QC without much need of control system. This was done by D-wave for the TSP problem and is the core of adiabatic quantum systems. However, the case can be made for conventional HPC requirement since QCs at present perform poorly on certain class of easy problems, which can become the systems primary bottleneck. Integrating conventional HPCs with QCs would mitigate such bottlenecks to a degree. However, the paper does not provide such an analysis.

2.4 Performance Metrics

This section begins with discussing limitations of conventional metrics such as FLOPS for the HPC-QC system. The authors argue that quantum FLOPS analogy being poorly defined for two main reasons. Firstly, the clock speed of the QPU would not be directly proportional to the speed at which the gates are applied in the qALU for computation. This is since the quantum gates have varying speeds of execution and cannot be made arbitrarily fast. The second reason it that the error-correction and the program (or data)-specific fault tolerant protocols complicate the matter further by adding more gates and qubits into the picture.

The paper then recommends four performance metrics for the new architecture:

i) Relative overhead required by the fault tolerant instruction protocols for the ISA.
ii) QCU cycles per second with longest and the shortest duration gates within the QRAM serving as the worst and the best-case performance measures respectively.
iii) Confidence intervals, minimum number of samples, halting conditions etc. defined by the quantum algorithmic complexity statements.
iv) Performance of the interconnect in transferring data while maintaining entanglement states and coordination with registers.

The paper argues that both i) and ii) do not require reference to a particular technology as they rely on abstractions of the QRAM-QPU interfaces. For example, in both circuit driven and adiabatic quantum systems, gate duration and spread can be measured regardless of the ISA. Meanwhile, iii) arises due to the probabilistic nature of quantum algorithms. Here, the paper focuses on scenarios where the users only deal with high level languages and libraries while the statistical nature of algorithms are hidden from the user. In such cases, QPU performances are likely affected by some pre- and post-processing steps in recasting the problem into some statistical versions. Finally, since interconnect is a key component of the architecture, and a possible source of bottleneck, the paper addresses its performance as another benchmark. Since all HPC-QC platforms will likely have interconnect, this will be another architecture blind metric.

Analysis

We agree with the paper that standard conventional metrics such as FLOPS or FLOPS/watt are poorly defined for the HPC-QC system. This is mainly since the QC ISA is poorly defined at present. However, we believe that FLOPS can act as a crude yardstick for the evolving QC system with respect to conventional HPC. For example, with a series of tests conducted on D-wave V5 chip with 439 operational qubits, the chip obtained an approximate Linpack rating of 420 GFlops [8]. Meanwhile, the same chip when applied to the TSP problem produced a maximum of 1.5 petaFlops. The reader should note that such benchmarking was performed by an expert and has its limitations. Nonetheless, we believe that such crude metrics can still provide a good picture of the status of the evolution of the QC systems in the HPC domain.

The four metrics described by the paper are excellent starting points for performance analysis among the evolving QC-HPC systems. These metrics would be in-domain where we compare two QC-HPC systems rather than cross-domain where we compared QC-HPC with conventional HPC. While the paper argues for the usage of the four metrics, it does not provide the details of how to use them in detail. This again would be ISA specific. Additionally, no metric is proposed for comparing cross-domain performances effectively such as some alternative to FLOPS, etc.

Among the four metric proposed, iii) is novel metric for HPC. Although probabilistic metrics are used in PAC learning algorithms in AI and machine learning, such metrics have been used for algorithm analysis rather than system analysis. Hence iii) as a metric for system's performances maybe marking an end of the deterministic era in computation. One such implementation of iii) is the time-to-target (TTT) metric [9]. A $TTT_{q,p,N}$ is defined for a parameter q, a quantum annealing solver p (for example, a D-wave 2X quantum annealing processor) and the input N. Here, a target energy E_q is first defined by gathering many samples from the solver, charting energy distributions from such samples and identifying E_q from a

quantile q of the sample distribution. For each tuple (N,E_q,p) a samples-to-target (STT) is defined as the expected number of samples required by the solver p to attain an energy at or below the E_q. A solver's TTT is the expected time required by the solver to reach E_q utilizing STT and computation times. For specific task of annealing, timing can be broken down to programming/initialization time (t_i), anneal time (t_a), and readout time (t_r). Let p_t be the probability that a single sample will reach the target energy. Then $\widehat{p_t}$ = 1/STT where $\widehat{p_t}$ is the mean p_t. Then, $TTT = \frac{t_i}{\widehat{p_g}} + \frac{t_a + t_r}{\widehat{p_t}}$ where $\widehat{p_g}$ a sample statistic for single gauge transformations in the solver [9]. Rewriting $TTT = at_i + bt_a + ct_r$ shows that the timing metric is a linear combination of initialization time, anneal time and the readout time which agrees with the papers argument that QPU performances will be affected by pre- and post-processing steps in reformulating a problem into a statistical version.

3. Summary

Britt et. al.'s paper on HPC-QC integration outlines a simple QPU network scheme and investigates integration of such a scheme into HPC platforms. For this, the paper investigates tight and loose integration pathways that differ mainly in the communication infrastructure between the host QPU and the client CPUs. The paper outlines the idea of the quantum interconnect and its application in the two integration paths. Finally, the paper provides additional metrics for benchmarking HPC-QC systems for which the existing metrics such as FLOPS would be insufficient.

We found merits and demerits in paper's generic approach to HPC-QC integration. The generic approach made the paper easy to comprehend and the proposed integration and benchmarking methods applicable in architecture agnostic settings. The generic approach, however, also made the paper highly simplistic. The paper would have benefitted from analyzing current quantum architectures in some detail and providing a practical basis of why their recommendations would work and on what scenario would they likely fail. In this paper, we dug into the "whys" of certain recommendations that the paper failed to provide. For this, we compared and analyzed the paper's recommendations against current systems such as concrete QRAM model, D-wave V5 chips, TTT benchmarking measure, etc. In doing so, we understood the core concepts put forward by the paper in the light of theoretical limitations, scope and some state-of-the-art implementations of the quantum computing systems.

References

1) Britt, et. al. "High-Performance Computing with Quantum Processing Units", 2015.
2) Lucas, et. al. "Top Ten Exascale Research Challenges" 2014
3) in El houshi et. al. "Modeling a Quantum Processor using the QRAM Model", 2011.
4) H. J. Kimble, "The Quantum Internet", 2008.
5) Spiller et. Al. "Quantum Computation by Communication", 2006
6) "A New Quantum Teleportation Distance Record Has Been Set", Fiona MacDonald 24 Sep 2015.
7) Eric Ladizinsky: "Evolving Scalable Quantum Computers", GoogleTechTalks, March 5, 2014.
8) Brian Dodson. "D-Wave quantum computer matches the tenth ranked supercomputer for speed". May 13, 2013.
9) King et. al. "Benchmarking a quantum annealing processor with the time-to-target metric.", August 20, 2015.